Desteklenen diğer veritabanları, işlevsel bağımlılığı algılayabildikleri için yukarıdaki örnekteki OperationalError hatasıyla karşılaşmazlar. Genel olarak AnyValue, veritabanı tarafından gruplama yan tümcesindeki sütunlara işlevsel olarak bağımlı olarak tanınmayan, toplama işlevleri içermeyen veya karmaşık ifadeler barındıran seçim listesi sütunlarıyla çalışırken kullanışlıdır.

AnyValue toplama işlevi eklendi.

3.2.4 Arama

Web uygulamaları için yaygın bir görev, kullanıcı girdisiyle veritabanındaki bazı verileri aramaktır. Basit bir durumda bu, bir nesne listesini kategoriye göre filtrelemek olabilir. Daha karmaşık bir kullanım durumu ise ağırlıklandırma, kategorize etme, vurgulama, çoklu dil vb. ile arama yapmayı gerektirebilir. Bu belge, olası kullanım durumlarından bazılarını ve kullanabileceğiniz araçları açıklamaktadır.

Sorgu oluşturma bölümünde kullanılan modellerden bahsedeceğiz.

Kullanım Senaryoları

Standart metin sorguları

Metin tabanlı alanlar için çeşitli eşleştirme işlemleri mevcuttur. Örneğin, bir yazarı şu şekilde aramak isteyebilirsiniz:

```
>>> Author.objects.filter(name contains="Terry")
[<Author: Terry Gilliam>, <Author: Terry Jones>]
```

Bu, kullanıcının yazar adının tam alt dizesini bilmesini gerektirdiği için çok kırılgan bir çözümdür. Daha iyi bir yaklaşım, büyük/küçük harf duyarlı olmayan eşleştirme (*icontains*) olabilir, ancak bu da sadece biraz daha iyidir.

Veritabanının daha gelişmiş karşılaştırma işlevleri

PostgreSQL kullanıyorsanız, Django daha karmaşık sorgulama seçeneklerinden yararlanmanızı sağlayan veritabanına özgü bir dizi araç sunar. Diğer veritabanları, muhtemelen eklentiler veya kullanıcı tanımlı işlevler aracılığıyla farklı araç seçeneklerine sahiptir. Django şu anda bunlar için herhangi bir destek sunmamaktadır. Veritabanlarının sahip olabileceği işlevselliği göstermek için PostgreSQL'den bazı örnekler kullanacağız.

> **Diğer veritabanlarında arama**
>
> *django.contrib.postgres* tarafından sağlanan tüm arama araçları, özel aramalar ve veritabanı işlevleri gibi tamamen genel API'ler üzerine inşa edilmiştir. Veritabanınıza bağlı olarak, benzer API'lere izin verecek sorgular oluşturabilmeniz gerekir. Bu şekilde gerçekleştirilemeyen belirli şeyler varsa, lütfen bir bilet açın.

Yukarıdaki örnekte, büyük/küçük harf duyarlı olmayan bir arama işleminin daha yararlı olacağına karar verdik. İngilizce olmayan isimlerle çalışırken, *aksansız karşılaştırma* kullanmak daha da iyi bir sonuç verir:

```
>>> Author.objects.filter(name unaccent icontains="Helen")
[<Author: Helen Mirren>, <Author: Helena Bonham Carter>, <Author: Hélène Joy>]
```

Bu, ismin farklı yazılışlarıyla eşleştirme yaptığımız başka bir sorunu göstermektedir. Ancak bu durumda bir asimetri söz konusudur: Helen araması Helena veya Hélène'i bulur, ancak tersi geçerli değildir. Başka bir seçenek, harf dizilerini karşılaştıran *trigram_similar* karşılaştırmasını kullanmaktır.

Örneğin:

```
>>> Author.objects.filter(name unaccent lower trigram_similar="Hélène")
[<Author: Helen Mirren>, <Author: Hélène Joy>]
```

Şimdi farklı bir sorunla karşı karşıyayız: "Helena Bonham Carter" gibi daha uzun isimler, çok daha uzun oldukları için sonuçlarda görünmüyor. Trigram aramaları, üç harfin tüm kombinasyonlarını dikkate alır ve arama dizesi ile kaynak dizede kaç tanesinin ortaya çıktığını karşılaştırır. Daha uzun isimler için, kaynak dizede görünmeyen daha fazla kombinasyon olduğundan, artık yakın eşleşme olarak değerlendirilmezler.

Burada doğru karşılaştırma fonksiyonunun seçimi, kullandığınız veri setine, örneğin kullanılan dil(ler)e ve aranan metnin türüne bağlıdır. Gördüğümüz tüm örnekler, kullanıcının kaynak verilere (çeşitli tanımlara göre) yakın bir şey girmesi muhtemel olan kısa dizeler üzerinedir.

Belge tabanlı arama

Büyük metin bloklarını ele almaya başladığınızda, standart veritabanı işlemleri artık yararlı bir yaklaşım olmaktan çıkar. Yukarıdaki örnekler bir karakter dizisi üzerinde yapılan işlemler olarak düşünülebilirken, tam metin araması gerçek kelimelere bakar. Kullanılan sisteme bağlı olarak, aşağıdaki fikirlerden bazıları kullanılması muhtemeldir:

- "a", "the", "and" gibi "durdurma kelimelerini" yok sayma.
- Kelimeleri kökleştirerek, "pony" ve "ponies" kelimelerinin benzer kabul edilmesini sağlamak.
- Kelimeleri, metinde ne sıklıkla göründükleri veya başlık ya da anahtar kelimeler gibi göründükleri alanların önemi gibi farklı kriterlere göre ağırlıklandırmak.

Arama yazılımı kullanmak için birçok alternatif vardır; en öne çıkanlardan bazıları Elastic ve Solr'dur. Bunlar, tam belge tabanlı arama çözümleridir. Bunları Django modellerinden gelen verilerle kullanmak için, verilerinizi veritabanı kimliklerine geri referanslar da dahil olmak üzere metin belgesine dönüştüren bir katmana ihtiyacınız olacaktır. Arama motorunu kullanarak yapılan bir arama belirli bir belgeyi döndürdüğünde, bu belgeyi veritabanında arayabilirsiniz. Bu sürece yardımcı olmak için tasarlanmış çeşitli üçüncü taraf kütüphaneler bulunmaktadır.

PostgreSQL desteği

PostgreSQL'in kendi yerleşik tam metin arama uygulaması vardır. Diğer bazı arama motorları kadar güçlü olmasa da, veritabanınızın içinde olması avantajı vardır ve bu sayede kategorizasyon gibi diğer ilişkisel sorgularla kolayca birleştirilebilir.

django.contrib.postgres modülü, bu sorguları oluşturmak için bazı yardımcı araçlar sağlar. Örneğin, bir sorgu "cheese" kelimesini içeren tüm blog girişlerini seçebilir:

```
>>> Entry.objects.filter(body_text search="cheese")
[<Entry: Cheese on Toast recipes>, <Entry: Pizza recipes>]
```

Ayrıca, alanların bir kombinasyonuna veilgili modellere göre de filtreleme yapabilirsiniz:

```
>>> Entry.objects.annotate(
...         search=SearchVector("blog tagline", "body_text"),
... ).filter(search="cheese") [
    <Entry: Cheese on Toast recipes,
    <Entry: Pizza Recipes>,
    <Entry: Dairy farming in Argentina>,l
]
```

Tüm ayrıntılar için `contrib.postgres` Tam metin arama belgesine bakın.

3.2.5 Yöneticiler

class Yönetici

Yönetici, Django modellerine veritabanı sorgu işlemlerinin sağlandığı arayüzdür. Bir Django uygulamasındaki her model için en az bir Yönetici bulunur.

Yönetici sınıflarının çalışma şekli Sorgu oluşturma bölümünde belgelenmiştir; bu belge özellikle Yönetici davranışını özelleştiren model seçeneklerine değinmektedir.

Yönetici adları

Varsayılan olarak, Django her Django model sınıfına objects adında bir Yönetici ekler. Ancak, objects'ı bir alan adı olarak kullanmak istiyorsanız veya Yönetici için objects dışında bir ad kullanmak istiyorsanız, bunu model bazında yeniden adlandırabilirsiniz. Belirli bir sınıfın Yöneticisini yeniden adlandırmak için, o modelde `models.Manager()` türünde bir sınıf özniteliği tanımlayın. Örneğin:

```
from django.db import models


class Person(models.Model):
    #...
    people = models.Manager()
```

Bu örnek modeli kullanarak, `Person.objects` bir `AttributeError` istisnası oluşturacaktır, ancak `Person.people.all()` tüm `Person` nesnelerinin bir listesini sağlayacaktır.

Özel yöneticiler

Belirli bir modelde özel bir Yönetici kullanmak için, temel Yönetici sınıfını genişletip modelinizde özel Yönetici'nizin bir örneğini oluşturabilirsiniz.

Bir Yöneticiyi özelleştirmek isteyebileceğiniz iki neden vardır: ekstra Yönetici yöntemleri eklemek ve/veya Yönetici'nin döndürdüğü ilk QuerySet'i değiştirmek.

Ekstra yönetici yöntemleri ekleme

Ekstra Manager yöntemleri eklemek, modellerinize "tablo düzeyinde" işlevsellik eklemenin tercih edilen yoludur. ("Satır düzeyinde" işlevsellik için, yani bir model nesnesinin tek bir örneğine etki eden işlevler için, özel Manager yöntemlerini değil, Model yöntemlerini kullanın.

Örneğin, bu özel `Manager with_counts()` yöntemini ekler:

```
from django.db import models
from django.db.models.functions import Coalesce


class PollManager(models.Manager):
    def with_counts(self):
        return self.annotate(num_responses=Coalesce(models.Count("response"), 0))


class OpinionPoll(models.Model):
    question = models.CharField(max_length=200)
    objects = PollManager()


class Response(models.Model):
    poll = models.ForeignKey(OpinionPoll, on_delete=models.CASCADE)
    #...
```

Bu örnekte, OpinionPoll.objects.with_counts() işlevini kullanarak, ek `num_responses` özniteliği eklenmiş OpinionPoll nesnelerinin QuerySet'ini elde edersiniz.

Özel bir Manager yöntemi, istediğiniz herhangi bir şeyi döndürebilir. `QuerySet` döndürmesi gerekmez.

Dikkat edilmesi gereken bir diğer nokta da, Manager yöntemlerinin `self.model'e` erişerek kendilerinin bağlı olduğu model sınıfını alabilmeleridir.

Bir yöneticinin ilk QuerySet'ini değiştirme

Bir Manager'ın temel QuerySet'i, sistemdeki tüm nesneleri döndürür. Örneğin, şu modeli kullanarak:

```
from django.db import models


class Book(models.Model):
    title = models.CharField(max_length=100)
    author = models.CharField(max_length=50)
```

. . . Book.objects.all() ifadesi, veritabanındaki tüm kitapları döndürür.

Manager.get_queryset() yöntemini geçersiz kılıp bir Manager'ın temel QuerySet'ini geçersiz kılabilirsiniz. get_queryset(), istediğiniz özelliklere sahip bir QuerySet döndürmelidir.

Örneğin, aşağıdaki modelde iki Yönetici vardır: biri tüm nesneleri döndüren, diğeri ise yalnızca Roald Dahl'ın kitaplarını döndüren.

Manager.get_queryset()

```
# Öncelikle, Manager alt sınıfını tanımlayalım.
class DahlBookManager(models.Manager): def
    get_queryset(self):
        return super().get_queryset().filter(author="Roald Dahl")



# Ardından bunu Book modeline açıkça bağlayın.
class Book(models.Model):
    title = models.CharField(max_length=100) author
    = models.CharField(max_length=50)

    objects = models.Manager()      #Varsayılan yönetici.
    dahl_objects = DahlBookManager()     # Dahl'a özgü yönetici.
```

Bu örnek modelde, Book.objects.all() veritabanındaki tüm kitapları döndürür, ancak Book.dahl_objects.all() yalnızca Roald Dahl tarafından yazılanları döndürür.

get_queryset() bir QuerySet nesnesi döndürdüğü için, üzerinde filter()exclude() ve diğer tüm QuerySet yöntemlerini kullanabilirsiniz. Dolayısıyla şu ifadelerin hepsi geçerlidir:

```
Book.dahl_objects.all()
Book.dahl_objects.filter(title="Matilda")
Book.dahl_objects.count()
```

Bu örnek, aynı model üzerinde birden fazla yönetici kullanma gibi ilginç bir tekniğe de işaret etmektedir. Bir modele istediğiniz kadar Manager() örneği ekleyebilirsiniz. Bu, modelleriniz için

örneğin:

```
class AuthorManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(role="A")


class EditorManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(role="E")


class Person(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    role = models.CharField(max_length=1, choices={"A": _("Author"), "E": _("Editor")})
    people = models.Manager()
    authors= AuthorManager()
    editors = EditorManager()
```

Bu örnek, `Person.authors.all()`, `Person.editors.all()` ve `Person.people.all()` isteklerini yapmanıza ve öngörülebilir sonuçlar almanıza olanak tanır.

Varsayılan yöneticiler

`Model._default_manager`

Özel `Yönetici` nesneleri kullanıyorsanız, Django'nun karşılaştığı ilk Yönetici'nin (modelde tanımlandıkları sırayla) özel bir statüsü olduğunu unutmayın. Django, bir sınıfta tanımlanan ilk Manager nesnesini "varsayılan" Manager olarak yorumlar ve Django'nun çeşitli kısımları (*dumpdata* dahil) o model için yalnızca bu Manager'ı kullanır. Sonuç olarak, get_queryset() işlevini geçersiz kılmanın, üzerinde çalışmak istediğiniz nesneleri alamamanıza yol açmasını önlemek için varsayılan yönetici seçiminizde dikkatli olmanız iyi bir fikirdir.

Meta.default_manager_name kullanarak özel bir varsayılan yönetici belirtebilirsiniz.

Örneğin, genel bir görünümü uygulayan bir üçüncü taraf uygulamada, bilinmeyen bir modeli işlemek zorunda olan bir kod yazıyorsanız, modelin bir nesne yöneticisine sahip olduğunu varsaymak yerine bu yöneticiyi (veya *_base_manager*) kullanın.

Temel yöneticiler

Model._base_manager

İlişkili nesnelere erişim için yöneticileri kullanma

Varsayılan olarak, Django ilişkili nesnelere (ör. choice.question) erişirken ilişkili nesnedeki _default_manager'ı değil, Model._base_manager yönetici sınıfının bir örneğini kullanır. Bunun nedeni, Django'nun, varsayılan yönetici tarafından filtrelenip (ve dolayısıyla erişilemez hale gelse bile) ilişkili nesneyi alabilmesi gerektiğidir.

Normal temel yönetici sınıfı (*django.db.models.Manager*) durumunuza uygun değilse, *Meta.base_manager_name* ayarını yaparak Django'ya hangi sınıfı kullanacağını belirtebilirsiniz.

İlişkili modellerde sorgu yaparken veya bir-çok ya da çok-çok ilişkisine erişirken temel yöneticiler kullanılmaz. Örneğin, öğreticideki Question modelinde bir deleted alanı ve deleted=True olan örnekleri filtreleyen bir temel yönetici varsa, Choice.objects.filter(question__name__startswith='What') gibi bir sorgu kümesi, silinmiş sorularla ilgili seçenekleri içerecektir.

Bu tür yönetici alt sınıflarında hiçbir sonucu filtrelemeyin

Bu yönetici, başka bir modelle ilişkili nesnelere erişmek için kullanılır. Bu durumlarda, Django, getirilen modelin tüm nesnelerini görebilmelidir, böylece atıfta bulunulan her şey alınabilir.

Bu nedenle, herhangi bir satırı filtrelemek için get_queryset() yöntemini geçersiz kılmamalısınız. Bunu yaparsanız, Django eksik sonuçlar döndürür.

Yönetici sınıfından özel QuerySet yöntemlerini çağırma

Standart QuerySet'teki yöntemlerin çoğuna Manager üzerinden doğrudan erişilebilirken, özel bir QuerySet üzerinde tanımlanan ek yöntemler için bu durum, yalnızca bunları Manager üzerinde de uyguladığınızda geçerlidir:

```
class PersonQuerySet(models.QuerySet):
    def authors(self):
        return self.filter(role="A")

    def editors(self):
        return self.filter(role="E")


class PersonManager(models.Manager):
  def get_queryset(self):
        PersonQuerySet(self.model, using=self._db) döndür
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    def authors(self):
        return self.get_queryset().authors()

    def editörler(self):
        return self.get_queryset().editors()


class Person (models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    role = models.CharField(max_length=1, choices={"A": _("Author"), "E": _("Editor")})
    people = PersonManager()
```

Bu örnek, `authors()` ve `editors()` işlevlerini doğrudan `Person.people` yöneticisinden çağırmanıza olanak tanır.

`QuerySet` *yöntemleriyle bir yönetici oluşturma*

`QuerySet` ve `Manager` üzerinde yöntemlerin çoğaltılmasını gerektiren yukarıdaki yaklaşımın yerine, *QuerySet.as_manager()* kullanılarak özel bir QuerySet'in yöntemlerinin bir kopyasını içeren bir Manager örneği oluşturulabilir:

```
class Person(models.Model):
    ...
    people = PersonQuerySet.as_manager()
```

QuerySet.as_manager() tarafından oluşturulan Manager örneği, önceki örnekteki `PersonManager` ile neredeyse aynı olacaktır.

Her QuerySet yöntemi, Manager düzeyinde anlamlı değildir; örneğin, *QuerySet.delete()* yönteminin Manager sınıfına kopyalanmasını kasıtlı olarak engelliyoruz.

Yöntemler aşağıdaki kurallara göre kopyalanır:

- Genel yöntemler varsayılan olarak kopyalanır.
- Özel yöntemler (alt çizgiyle başlayanlar) varsayılan olarak kopyalanmaz.
- queryset_only özniteliği False olarak ayarlanmış yöntemler her zaman kopyalanır.
- queryset_only özniteliği True olarak ayarlanmış yöntemler asla

kopyalanmaz. Örneğin:

```
class CustomQuerySet(models.QuerySet):
    # Hem Manager hem de QuerySet'te kullanılabilir.
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    def public_method(self): return

    # Yalnızca QuerySet'te kullanılabilir.
    def _private_method(self):
        return

    # Yalnızca QuerySet'te kullanılabilir.
    def opted_out_public_method(self): return

    opted_out_public_method.queryset_only = True

    # Hem Manager hem de QuerySet'te kullanılabilir.
    def _opted_in_private_method(self):
        return

    _opted_in_private_method.queryset_only = False
```

from_queryset()

classmethod from_queryset(queryset_class)

Gelişmiş kullanım için hem özel bir Manager hem de özel bir QuerySet isteyebilirsiniz. Bunu, özel QuerySet yöntemlerinin bir kopyasını içeren temel Manager sınıfınızın bir alt sınıfını döndüren Manager.from_queryset() işlevini çağırarak yapabilirsiniz:

```
class CustomManager(models.Manager):
    def manager_only_method(self):
        return


class CustomQuerySet(models.QuerySet): def
    manager_and_queryset_method(self):
        return


class MyModel(models.Model):
    objects = CustomManager.from_queryset(CustomQuerySet)()
```

Oluşturulan sınıfı bir değişkene de kaydedebilirsiniz:

```
MyManager = CustomManager.from_queryset(CustomQuerySet)


class MyModel(models.Model):
    objects = MyManager()
```

Özel yöneticiler ve model kalıtımı

Django'nun özel yöneticileri ve model kalıtımını nasıl işlediği aşağıda açıklanmıştır:

1. Temel sınıflardaki yöneticiler, Python'un normal isim çözümleme sırası kullanılarak her zaman alt sınıf tarafından devralınır (alt sınıftaki isimler diğerlerinin hepsini geçersiz kılar; ardından ilk üst sınıftaki isimler gelir ve bu şekilde devam eder).
2. Bir modelde ve/veya üst sınıflarında yönetici tanımlanmamışsa, Django otomatik olarak yöneticisini oluşturur.
3. Bir sınıftaki varsayılan yönetici, *Meta.default_manager_name* ile seçilen yönetici, modelde bildirilen ilk yönetici veya ilk üst modelin varsayılan yöneticisidir.

Bu kurallar, soyut bir temel sınıf aracılığıyla bir grup modele özel yönetici koleksiyonu yüklemek, ancak yine de varsayılan yöneticiyi özelleştirmek istediğinizde gerekli esnekliği sağlar. Örneğin, şu temel sınıfa sahip olduğunuzu varsayalım:

```
class AbstractBase(models.Model):
    #...
     objects = CustomManager()

    class Meta:
        abstract = True
```

Bunu bir alt sınıfta doğrudan kullanırsanız, alt sınıfta herhangi bir yönetici tanımlamazsanız varsayılan `objects` yönetici olacaktır:

```
class ChildA(AbstractBase):
    # ...
    # Bu sınıfın varsayılan yöneticisi CustomManager'dır.
    pass
```

AbstractBase'den miras almak, ancak farklı bir varsayılan yönetici sağlamak istiyorsanız, varsayılan yöneticiyi alt sınıfta sağlayabilirsiniz:

```
class ChildB(AbstractBase):
    #...
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    # Açık bir varsayılan yönetici.
    default_manager = OtherManager()
```

Burada, default_manager varsayılan değerdir. Nesne yöneticisi miras alındığı için hala kullanılabilir durumdadır, ancak varsayılan olarak kullanılmaz.

Son olarak, bu örnek için, alt sınıfa ekstra yöneticiler eklemek, ancak yine de AbstractBase'den gelen varsayılanı kullanmak istediğinizi varsayalım. Yeni yöneticiyi doğrudan alt sınıfa ekleyemezsiniz, çünkü bu varsayılanı geçersiz kılar ve ayrıca soyut temel sınıftaki tüm yöneticileri açıkça dahil etmeniz gerekir. Çözüm, ekstra yöneticileri başka bir temel sınıfa koymak ve bunları varsayılanlardan sonra miras hiyerarşisine dahil etmektir:

```
class ExtraManager(models.Model):
extra_manager = OtherManager()

    class Meta:
        abstract = True



class ChildC(AbstractBase, ExtraManager):
    # ...
    # Varsayılan yönetici CustomManager'dır, ancak OtherManager
    # "extra_manager" özniteliği aracılığıyla da kullanılabilir.
    pass
```

Soyut model üzerinde özel bir yönetici tanımlayabilirsiniz, ancak soyut modeli kullanarak herhangi bir yöntemi çağıramazsınız. Yani:

```
ClassA.objects.do_something()
```

geçerlidir, ancak:

```
AbstractBase.objects.do_something()
```

bir istisna oluşturacaktır. Bunun nedeni, yöneticilerin nesne koleksiyonlarını yönetmek için mantığı kapsüllemeyi amaçlamasıdır. Soyut nesnelerden oluşan bir koleksiyona sahip olamayacağınız için, bunları yönetmenin bir anlamı yoktur. Soyut modele uygulanan bir işlevselliğiniz varsa, bu işlevselliği soyut modeldeki bir staticmethod veya classmethod içine koymalısınız.

Uygulama ile ilgili hususlar

Özel Manager'ınıza hangi özellikleri eklerseniz ekleyin, bir Manager örneğinin sığ bir kopyasını oluşturmak mümkün olmalıdır; yani, aşağıdaki kod çalışmalıdır:

```
>>> import copy
>>> manager = MyManager()
>>> my_copy = copy.copy(manager)
```

Django, belirli sorgular sırasında yönetici nesnelerinin yüzeysel kopyalarını oluşturur; eğer yöneticiniz kopyalanamıyorsa, bu sorgular başarısız olur.

Bu, çoğu özel yönetici için bir sorun oluşturmaz. Manager'ınıza sadece basit yöntemler ekliyorsanız, Manager'ınızın örneklerini istemeden kopyalanamaz hale getirmeniz olası değildir. Ancak, Manager nesnenizin durumunu kontrol eden getattr ____ veya başka bir özel yöntemi geçersiz kılıyorsanız, Manager'ınızın kopyalanabilirlik özelliğini etkilemediğinizden emin olmalısınız.

3.2.6 Ham SQL sorguları gerçekleştirme

Django, ham SQL sorguları gerçekleştirmek için üç yol sunar: *RawSQL'i* kullanarak ham SQL parçalarını ORM sorgularına gömebilir (bkz. Ham SQL parçaları), *Manager.raw()* işlevini kullanarak ham sorgular gerçekleştirebilir ve model örnekleri döndürebilir ya da model katmanını tamamen atlayarak özel SQL sorgularını doğrudan çalıştırabilirsiniz.

> **Ham SQL kullanmadan önce ORM'yi keşfedin!**
>
> Django ORM, ham SQL yazmadan sorguları ifade etmek için birçok araç sağlar. Örneğin:
>
> - QuerySet API'si oldukça kapsamlıdır.
> - Birçok yerleşik veritabanı işlevini kullanarak *açıklama ekleyebilir* ve toplama işlemleri yapabilirsiniz. Bunların ötesinde, özel sorgu ifadeleri oluşturabilirsiniz.
>
> Raw SQL kullanmadan önce ORM'yi keşfedin. ORM'nin kullanım durumunuzu destekleyip desteklemediğini öğrenmek için destek kanallarından birine danışın.

> **Uyarı**
>
> Ham SQL yazarken çok dikkatli olmalısınız. Her kullandığınızda, SQL enjeksiyon saldırılarına karşı korunmak için params kullanarak kullanıcının kontrol edebileceği tüm parametreleri uygun şekilde kaçırmalısınız. Lütfen SQL enjeksiyon koruması hakkında daha fazla bilgi edinin.

Ham SQL parçaları

Bazı durumlarda, ham SQL parçalarını doğrudan ORM sorgularına gömmeniz gerekebilir — örneğin, *annotate()* veya *filter()* çağrılarında. Arka uçlar arasında veritabanı işlevlerini çağırmak için Func() ifadeleri veya rastgele parametreli SQL parçaları için *RawSQL* kullanın.

Ham sorguların gerçekleştirilmesi

raw() yönetici yöntemi, model örnekleri döndüren ham SQL sorguları gerçekleştirmek için kullanılabilir:

Manager.raw(raw_query, params=(), translations=None)

Bu yöntem, ham bir SQL sorgusunu alır, onu yürütür ve bir django.db.models.query.RawQuerySet örneği döndürür. Bu RawQuerySet örneği, nesne örnekleri sağlamak üzere normal bir *QuerySet* gibi döngüye alınabilir.

Bu, bir örnekle en iyi şekilde açıklanabilir. Aşağıdaki modele sahip olduğunuzu varsayalım:

```
class Person(models.Model):
 first_name = models.CharField(...)
 last_name  = models.CharField(...)
 birth_date= models.DateField(...)
```

Böylece, aşağıdaki gibi özel bir SQL komutu çalıştırabilirsiniz:

```
>>> for p in Person.objects.raw("SELECT * FROM myapp_person"):
...         print(p)
...
John Smith
Jane Jones
```

Bu örnek pek heyecan verici değil – Person.objects.all() komutunu çalıştırmakla tamamen aynı. Ancak, raw(), onu çok güçlü kılan bir dizi başka seçeneğe sahiptir.

> **Model tablo adları**
>
> Bu örnekte Person tablosunun adı nereden geldi?
>
> Varsayılan olarak Django, modelin "uygulama etiketi"ni (manage.py startapp komutunda kullandığınız ad) modelin sınıf adıyla birleştirerek ve aralarına bir alt çizgi ekleyerek veritabanı tablo adını belirler.

örnekte, Person modelinin myapp adlı bir uygulamada yer aldığını varsaydık, bu nedenle tablosu myapp_person olur.

Daha fazla ayrıntı için, veritabanı tablosu adını manuel olarak ayarlamanıza da olanak tanıyan db_table seçeneğinin belgelerine göz atın.

Uyarı

.raw() işlevine geçirilen SQL deyimi üzerinde herhangi bir kontrol yapılmaz. Django, deyimin veritabanından bir dizi satır döndürmesini bekler, ancak bunu zorlamak için hiçbir şey yapmaz. Sorgu satır döndürmezse, (muhtemelen anlaşılması zor) bir hata oluşur.

Uyarı

MySQL üzerinde sorgu yapıyorsanız, MySQL'in sessiz tip zorlamasının, tipleri karıştırdığınızda beklenmedik sonuçlara neden olabileceğini unutmayın. Bir string tipi sütunda, ancak bir tamsayı değeriyle sorgu yaparsanız, MySQL karşılaştırmayı gerçekleştirmeden önce tablodaki tüm değerlerin tiplerini bir tamsayıya zorlayacaktır. Örneğin, tablonuzda 'abc', 'def' değerleri varsa ve WHERE mycolumn=0 sorgusu yaparsanız, her iki satır da eşleşecektir. Bunu önlemek için, değeri sorguda kullanmadan önce doğru tür dönüştürme işlemini gerçekleştirin.

Sorgu alanlarını model alanlarına eşleme

raw(), sorgudaki alanları otomatik olarak modeldeki alanlara eşler.

Sorgudaki alanların sırası önemli değildir. Diğer bir deyişle, aşağıdaki iki sorgu da aynı şekilde çalışır:

```
>>> Person.objects.raw("SELECT id, first_name, last_name, birth_date FROM myapp_person") >>>
Person.objects.raw("SELECT last_name, birth_date, first_name, id FROM myapp_person")
```

Eşleştirme, ada göre yapılır. Bu, sorgudaki alanları model alanlarına eşlemek için SQL'in AS cümlelerini kullanabileceğiniz anlamına gelir. Dolayısıyla, içinde Person verileri bulunan başka bir tablonuz varsa, bunu Person örneklerine kolayca eşleyebilirsiniz:

```
>>> Person.objects.raw("""
...     SELECT first AS first_name,
...            last AS last_name,
...            bd AS birth_date,
...            pk AS id,
...     FROM some_other_table
...     """)
```

Adlar eşleştiği sürece, model örnekleri doğru şekilde oluşturulacaktır.

Alternatif olarak, raw() işlevinin translations argümanını kullanarak sorgudaki alanları model alanlarına eşleştirebilirsiniz. Bu, sorgudaki alanların adlarını modeldeki alanların adlarına eşleştiren bir sözlüktür. Örneğin, yukarıdaki sorgu şu şekilde de yazılabilir:

```
>>> name_map = {"first": "first_name", "last": "last_name", "bd": "birth_date", "pk": "id
↪"}
>>> Person.objects.raw("SELECT * FROM some_other_table", translations=name_map)
```

Dizin aramaları

raw() indekslemeyi destekler, bu nedenle yalnızca ilk sonuca ihtiyacınız varsa şunu yazabilirsiniz:

```
>>> first_person = Person.objects.raw("SELECT * FROM myapp_person")[0]
```

Ancak, indeksleme ve dilimleme işlemleri veritabanı düzeyinde gerçekleştirilmez. Veritabanınızda çok sayıda Person nesnesi varsa, sorguyu SQL düzeyinde sınırlamak daha verimlidir:

```
>>> first_person = Person.objects.raw("SELECT * FROM myapp_person LIMIT 1")[0]
```

Model alanlarını erteleme

Alanlar da dışarıda bırakılabilir:

```
>>> people = Person.objects.raw("SELECT id, first_name FROM myapp_person")
```

Bu sorgu tarafından döndürülen Person nesneleri, ertelenmiş model örnekleri olacaktır (bkz. *defer()*). Bu, sorgudan çıkarılan alanların talep üzerine yükleneceği anlamına gelir. Örneğin:

```
>>> for p in Person.objects.raw("SELECT id, first_name FROM myapp_person"):
...         print(
...             p.first_name, # Bu, orijinal sorgu tarafından alınacaktır
...             p.last_name, # Bu, talep üzerine alınacaktır
...         )
...
John Smith
Jane Jones
```

Dışarıdan bakıldığında, bu sorgu hem adı hem de soyadı getirmiş gibi görünüyor. Ancak, bu örnekte aslında 3 sorgu gönderilmiştir. raw() sorgusu sadece adları getirmiştir – soyadları ise yazdırıldıklarında talep üzerine getirilmiştir.

Hariç tutamayacağınız tek bir alan vardır: birincil anahtar alanı. Django, birincil anahtarı

, bu nedenle her zaman raw sorgusuna dahil edilmelidir. Birincil anahtarı eklemeyi unutursanız, bir *FieldDoesNotExist* istisnası ortaya çıkar.

raw() işlevine parametreler aktarma

Parametreli sorgular gerçekleştirmeniz gerekiyorsa, `raw()` işlevinde `params` argümanını kullanabilirsiniz:

```
>>> lname = "Doe"
>>> Person.objects.raw("SELECT * FROM myapp_person WHERE last_name = %s", [lname])
```

params, parametrelerin bulunduğu bir liste veya sözlüktür. Veritabanı motorunuzdan bağımsız olarak, sorgu dizesinde liste için %s yer tutucularını veya sözlük için %(key)s yer tutucularını (burada key, sözlük anahtarı ile değiştirilir) kullanacaksınız. Bu tür yer tutucular, params argümanındaki parametrelerle değiştirilecektir.

> **Not**
>
> Sözlük params, SQLite arka ucunda desteklenmez; bu arka uçta, parametreleri bir liste olarak geçirmelisiniz.

> **Uyarı**
>
> Ham sorgularda dize biçimlendirme kullanmayın veya SQL dizelerinizdeki yer tutucuları tırnak içine almayın! Yukarıdaki sorguyu şu şekilde yazmak cazip gelebilir:
>
> ```
> >>> query = "SELECT * FROM myapp_person WHERE last_name = %s" % lname
> >>> Person.objects.raw(query)
> ```
>
> Ayrıca sorgunuzu şu şekilde yazmanız gerektiğini düşünebilirsiniz (`%s'nin` etrafını tırnak işaretleriyle):
>
> ```
> >>>query ="SELECT * FROM myapp_person WHERE last_name = '%s'"
> ```
>
> Bu hatalardan hiçbirini yapmayın.
>
> SQL enjeksiyonu koruması bölümünde de belirtildiği gibi, params argümanını kullanmak ve yer tutucuları tırnak içine almamak, saldırganların veritabanınıza rastgele SQL kodları enjekte ettiği yaygın bir güvenlik açığı olan SQL enjeksiyon saldırılarına karşı koruma sağlar. Dize içe aktarma kullanırsanız veya yer tutucuyu tırnak içine alırsanız, SQL enjeksiyonu riskiyle karşı karşıya kalırsınız.

Özel SQL'i doğrudan yürütme

Bazen *Manager.raw()* bile yeterli olmaz: modellere tam olarak eşleşmeyen sorgular gerçekleştirmeniz veya UPDATE, INSERT veya DELETE sorgularını doğrudan çalıştırmanız gerekebilir.

Bu durumlarda, model katmanını tamamen atlayarak veritabanına her zaman doğrudan erişebilirsiniz.

django.db.connection nesnesi, varsayılan veritabanı bağlantısını temsil eder. Veritabanı bağlantısını kullanmak için

, connection.cursor() işlevini çağırarak bir imleç nesnesi elde edin. Ardından, cursor.execute(sql, [params]) işlevini çağırarak SQL'i çalıştırın ve cursor.fetchone() veya cursor.fetchall() işlevlerini çağırarak sonuç satırlarını döndürün.

Örneğin:

```
from django.db import connection



def my_custom_sql(self):
    with connection.cursor() as cursor:
        cursor.execute("UPDATE bar SET foo = 1 WHERE baz = %s", [self.baz])
        cursor.execute("SELECT foo FROM bar WHERE baz = %s", [self.baz]) row
        = cursor.fetchone()

    return row
```

SQL enjeksiyonuna karşı korunmak için, SQL dizesindeki %s yer tutucularının etrafına tırnak işareti koymamalısınız.

Sorguya gerçek yüzde işaretleri eklemek istiyorsanız, parametreleri aktarırken bunları iki kez yazmanız gerektiğini unutmayın:

```
cursor.execute("SELECT foo FROM bar WHERE baz = '30%'")
cursor.execute("SELECT foo FROM bar WHERE baz = '30%%' AND id = %s", [self.id])
```

Birden fazla veritabanı kullanıyorsanız, belirli bir veritabanı için bağlantıyı (ve imleci) elde etmek için django.db.connections kullanabilirsiniz. django.db.connections , takma adını kullanarak belirli bir bağlantıyı almanızı sağlayan sözlük benzeri bir nesnedir:

```
from django.db import connections

with connections["my_db_alias"].cursor() as cursor:
    #Kodunuzu buraya yazın
    ...
```

Varsayılan olarak, Python DB API sonuçları alan adları olmadan döndürür; bu da, bir dict yerine bir değerler listesi ile sonuçlanacağınız anlamına gelir. Küçük bir performans ve bellek maliyeti karşılığında, aşağıdaki gibi bir kod kullanarak sonuçları bir olarak döndürebilirsiniz:

```
def dictfetchall(cursor):
    """
    Bir imleçten tüm satırları bir dict olarak
    döndürür. Sütun adlarının benzersiz olduğunu
    varsayalım.
    """
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    columns = [col[0] for col in cursor.description]
    return [dict(zip(columns, row)) for row in cursor.fetchall()]
```

Diğer bir seçenek ise Python standart kütüphanesindeki collections.namedtuple() işlevini kullanmaktır. Namedtuple, öznitelik aramasıyla erişilebilen alanlara sahip, tuple benzeri bir nesnedir; ayrıca indekslenebilir ve yinelenebilir. Sonuçlar değişmez ve alan adları veya indekslerle erişilebilir, bu da yararlı olabilir:

```
from collections import namedtuple


def namedtuplefetchall(cursor): """
    Bir imleçten tüm satırları namedtuple olarak döndürür.
    Sütun adlarının benzersiz olduğunu varsayalım.
    """
    desc = cursor.description
    nt_result = namedtuple("Result", [col[0] for col in desc])
    return [nt_result(*row) for row in cursor.fetchall()]
```

dictfetchall() ve namedtuplefetchall() örnekleri, bir imleç farklı tablolardaki sütunları ayırt edemediği için sütun adlarının benzersiz olduğunu varsayar.

İşte bu üçünün arasındaki farkın bir örneği:

```
>>> cursor.execute("SELECT id, parent_id FROM test LIMIT 2") >>>
cursor.fetchall()
((54360982, None), (54360880, None))

>>> cursor.execute("SELECT id, parent_id FROM test LIMIT 2")
>>> dictfetchall(cursor)
[{'parent_id': None, 'id': 54360982}, {'parent_id': None, 'id': 54360880}]

>>> cursor.execute("SELECT id, parent_id FROM test LIMIT 2")
>>> results = namedtuplefetchall(cursor)
>>> results
[Result(id=54360982, parent_id=None), Result(id=54360880, parent_id=None)]
>>> results[0].id
54360982
>>> results [0][0]
54360982
```

Bağlantılar ve imleçler

bağlantı ve imleç, işlem yönetimi hariç olmak üzere, çoğunlukla PEP 249'da açıklanan standart Python DB-API'yi uygular.

Python DB-API'ye aşina değilseniz, cursor.execute() içindeki SQL deyiminin, parametreleri doğrudan SQL'e eklemek yerine "%s" yer tutucularını kullandığını unutmayın. Bu tekniği kullanırsanız, altta yatan veritabanı kütüphanesi parametrelerinizi gerektiği gibi otomatik olarak kaçıracaktır.

Ayrıca, Django'nun SQLite Python bağlamlarında kullanılan "?" yer tutucusu yerine "%s" yer tutucusunu beklediğini unutmayın. Bu, tutarlılık ve mantık açısından gereklidir.

Bir imleci bağlam yöneticisi olarak kullanma:

```
with connection.cursor() as c:
    c.execute(...)
```

şuna eşdeğerdir:

```
c = connection.cursor()
try:
    c.execute(...)
son olarak:
    c.close()
```

Saklı yordamları çağırma

CursorWrapper.callproc(procname, params=None, kparams=None)

Verilen isimle bir veritabanı saklı yordamını çağırır. Giriş parametrelerinin bir dizisi (params) veya sözlüğü (kparams) sağlanabilir. Çoğu veritabanı kparams'ı desteklemez. Django'nun yerleşik arka uçlarından sadece Oracle bunu destekler.

Örneğin, Oracle veritabanında şu saklı yordam olduğunu varsayalım:

```
CREATE PROCEDURE "TEST_PROCEDURE"(v_i INTEGER, v_text NVARCHAR2(10)) AS
    p_i INTEGER;
    p_text NVARCHAR2(10);
BEGIN
    p_i := v_i;
    p_text := v_text;
    ...
END;
```

Bu, onu şu şekilde çağıracaktır: